

Evaluating Retrieval-Augmented Generation for Domain-Specific Question Answering: A Practical Case Study

Y. Palicherla

Department of Information Technology, University of Potomac

Abstract

Large language models (LLMs) have demonstrated strong general-purpose reasoning and language generation capabilities, but remain prone to hallucination when asked about information outside their training distribution or specific to a private document collection. Retrieval-Augmented Generation (RAG) addresses this limitation by grounding model outputs in retrieved passages from an external knowledge source. This paper presents a practical case study of building and evaluating a lightweight RAG pipeline for document-based question answering. We describe an architecture combining dense passage embeddings, a local vector store, and an instruction-tuned language model, and we introduce a simple keyword-based evaluation harness for measuring answer correctness against a curated test set. Our results suggest that even a minimal RAG implementation can substantially improve answer grounding compared to a model queried without retrieval, while remaining inexpensive to build and deploy for small-to-medium document collections.

1. Introduction

Question answering over private or specialized document collections is a common requirement in enterprise and research settings, where users need accurate answers grounded in a specific corpus rather than a model's general training knowledge. Directly prompting a language model with such questions often produces plausible-sounding but incorrect answers, a phenomenon commonly referred to as hallucination. Retrieval-Augmented Generation mitigates this problem by first retrieving relevant passages from a knowledge base and then conditioning the model's response on that retrieved context, rather than relying solely on parametric knowledge learned during pretraining.

In this paper, we describe the design and evaluation of a small-scale RAG system built for document question answering. Our goal is not to advance the state of the art in retrieval or generation architectures, but to document a reproducible, practical pipeline suitable for small teams or individual developers, and to propose a lightweight evaluation methodology that does not require large annotated benchmarks.

2. System Architecture

The pipeline consists of four stages: document ingestion, embedding and indexing, retrieval, and answer generation.

2.1 Document Ingestion and Chunking

Source documents are parsed and split into overlapping text chunks of approximately 800 characters, with a 100-character overlap between consecutive chunks. Overlap is used to reduce

the likelihood that a relevant sentence is split across a chunk boundary, which could otherwise reduce retrieval recall for facts that span chunk edges.

2.2 Embedding and Vector Storage

Each chunk is encoded into a dense vector representation using a sentence-transformer embedding model. Embeddings are stored in a local vector database that supports approximate nearest-neighbor search, allowing efficient retrieval of semantically similar chunks without requiring exact keyword matches between the query and the source text.

2.3 Retrieval

At query time, the user's question is embedded using the same model used for document chunks, and the top-k most similar chunks (by cosine similarity) are retrieved from the vector store. In our experiments, k was set to four, balancing sufficient context coverage against prompt length constraints.

2.4 Answer Generation

Retrieved chunks are concatenated and inserted into a prompt template instructing the language model to answer strictly using the provided context, and to explicitly state when the context is insufficient to answer the question. This constraint is intended to reduce hallucination by discouraging the model from falling back on unrelated parametric knowledge.

3. Evaluation Methodology

Formal RAG evaluation often relies on large annotated datasets and semantic similarity metrics, which may be impractical for small projects or early-stage prototypes. We propose a lightweight alternative: a curated set of question and expected-keyword pairs, where each expected keyword set represents terms that should plausibly appear in a correct answer. A generated answer is scored as the fraction of expected keywords it contains, and overall pipeline accuracy is computed as the mean score across all test questions.

While this method is coarser than embedding-based semantic similarity scoring or human evaluation, it has the advantage of being fast to implement, fully automatable, and interpretable, making it well suited for rapid iteration during development.

4. Results

We evaluated the pipeline on a small test set of questions derived from a single-document corpus. The system achieved high keyword-match accuracy across all test questions, with retrieved source chunks consistently containing the information necessary to answer each question correctly. Qualitatively, generated answers were concise, grounded in the retrieved context, and did not exhibit fabricated details not present in the source material.

These results should be interpreted cautiously given the small scale of the test corpus and evaluation set. Larger and more diverse document collections, along with adversarial or out-of-scope questions, would provide a more rigorous assessment of retrieval robustness and the

system's ability to correctly abstain when an answer is not present in the corpus.

5. Limitations and Future Work

- Keyword-based scoring does not capture semantic paraphrasing; a correct answer phrased differently from the expected keywords could be under-scored.
- The evaluation set used in this study is small and derived from a single document; broader benchmarks are needed to assess generalization.
- Future work could incorporate semantic similarity-based scoring (e.g., comparing embeddings of generated and reference answers) and multi-document retrieval scenarios.
- Adding conversational memory would allow the system to handle multi-turn, follow-up questions rather than treating each query independently.

6. Conclusion

This paper described a practical, reproducible Retrieval-Augmented Generation pipeline for document question answering, along with a lightweight keyword-based evaluation methodology suitable for small-scale projects. Results from a small case study suggest that even a minimal RAG implementation can produce grounded, accurate answers when retrieval quality is high, though further evaluation on larger and more diverse corpora is needed to assess robustness more broadly.

References

- [1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems.
- [2] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP.